

Programación 1

Trabajo Practico Integrador

Alumnos:

Santiago Agüero y Liam Saez Aramune

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Docente Titular

Sofia Elizabeth Fernández

08 de Junio de 2026

Tabla de contenido

Marco Teórico	2
Decisiones Técnicas y Arquitectura	5
Desarrollo del proyecto	5
Diagrama de flujos principales	12
Dificultades y Conclusiones	13
Bibliografía / Webgrafía:	13
Links	14

Marco Teorico

Las siguiente estructuras fueron utilizadas para confección del presente proyecto:

1- Listas : estructuras de datos que permiten almacenar múltiples elementos en una única variable. En Python, las listas son ordenadas y modificables, lo que facilita agregar, eliminar o recorrer elementos mediante ciclos.

En este proyecto se utilizan listas para almacenar la información de los países cargados desde el archivo de datos. Por ejemplo, en la función `buscar_pais()`, se crea la lista `países`, donde cada elemento representa un país. Posteriormente, esta lista es recorrida para realizar búsquedas y otras operaciones.

<https://docs.python.org/es/3/tutorial/datastructures.html>

2-Diccionarios : Los diccionarios son estructuras de datos que almacenan información mediante pares clave-valor. Permiten acceder rápidamente a los datos utilizando una clave descriptiva en lugar de una posición numérica.

En el sistema, cada país se representa mediante un diccionario con las claves nombre, superficie, poblacion y continente. Esto permite acceder a los datos de forma clara y organizada.

<https://docs.python.org/es/3/tutorial/datastructures.html#dictionaries>

3-Funciones: permiten dividir un programa en bloques reutilizables de código, facilitando la organización, el mantenimiento y la legibilidad del sistema.

En este proyecto se implementaron distintas funciones para separar las responsabilidades del programa. Algunas de ellas son menu(), buscar_pais(), actualizar_paises(), filtrar_paises() y mostrar_estadisticas(). Cada función se encarga de realizar una tarea específica dentro del sistema.

<https://docs.python.org/es/3/tutorial/controlflow.html#defining-functions>

4-Condicionales

Permiten ejecutar diferentes bloques de código según se cumpla o no una condición determinada. En Python se implementan principalmente mediante las sentencias if, elif y else.

Dentro del programa, los condicionales se utilizan para validar opciones del menú, verificar que los campos ingresados no estén vacíos y controlar que los datos introducidos por el usuario sean correctos antes de procesarlos.

<https://docs.python.org/es/3/tutorial/controlflow.html#if-statements>

5-Ordenamientos

Los algoritmos de ordenamiento permiten reorganizar los datos según un criterio determinado, facilitando la visualización y el análisis de la información.

En este proyecto se implementó la posibilidad de ordenar los países por nombre, población o superficie, tanto de forma ascendente como descendente. Para ello se utilizan las funciones de ordenamiento proporcionadas por Python, aplicando criterios específicos sobre los datos almacenados.

<https://docs.python.org/es/3/howto/sorting.html>

6-Estadísticas básicas: permiten obtener información relevante a partir de un conjunto de datos, facilitando su interpretación y análisis.

El sistema calcula indicadores como el país con mayor población, el país con menor población, el promedio de población, el promedio de superficie y la cantidad de países por continente. Estos resultados permiten resumir la información almacenada en el dataset.

<https://docs.python.org/es/3/library/statistics.html>

7-Archivos CSV : formato ampliamente utilizado para almacenar información tabular mediante valores separados por comas.

En el proyecto se utiliza un archivo de texto con estructura CSV para almacenar los datos de los países. El programa puede leer la información desde el archivo, procesarla y utilizarla para realizar búsquedas, filtros y estadísticas. Además, los nuevos países ingresados por el usuario son almacenados de forma permanente dentro del archivo.

<https://docs.python.org/es/3/library/csv.html>

Decisiones Técnicas y Arquitectura

En cuanto a la arquitectura del proyecto, se decidió realizar una modularización en base a sus responsabilidades funcionales, organizando las funciones en distintos archivos .py según la operación que cumplen dentro del sistema.

Un archivo principal [main.py](#) desde el cual se llamará a las distintas funciones alojadas en distintos archivos a su mismo nivel pero organizadas en base a su funcionamiento, ya sea, ingreso de datos, actualización, búsqueda, filtros, orden, funciones auxiliares o procesamiento de datos para devolver estadísticas.

Esta organización responde al principio de que cada función debe cumplir una única responsabilidad, favoreciendo la legibilidad, el mantenimiento y la reutilización del código, además de facilitar el trabajo en equipo al permitir que cada integrante trabaje sobre módulos independientes sin generar conflictos.

Desarrollo del proyecto

El presente proyecto consiste en el desarrollo de una aplicación de consola en Python destinada a la gestión de información de países. El sistema permite almacenar, consultar, modificar y analizar datos relacionados con distintos países, utilizando estructuras de datos fundamentales de Python y persistencia de información mediante archivos de texto con formato CSV.

La aplicación fue estructurada siguiendo un enfoque modular, donde cada funcionalidad principal se implementa mediante una función independiente. Permitiendo mejorar la organización del código, facilitar su mantenimiento y favorecer la reutilización de sus distintos componentes.

Estructura de Datos Utilizada

La información de cada país se almacena mediante diccionarios de Python. Cada diccionario contiene los siguientes atributos: Nombre, Superficie, Población, Continente.

Todos los países se almacenan dentro de una lista denominada países, permitiendo a las distintas funciones poder: recorrer, buscar, filtrar y ordenar los registros de forma eficiente.

Persistencia de Datos

Para conservar la información entre ejecuciones del programa se utiliza un archivo de texto llamado “países.csv” el cual utiliza un formato CSV en el cual cada línea dentro del archivo contiene información respecto a un país.

La lectura de los datos se realiza mediante la función auxiliar `cargar_paises()`, que convierte cada línea del archivo en un diccionario y los almacena estos dentro de una lista

Menú Principal

El acceso a todas las funcionalidades se realiza desde la función `menu()`, que presenta al usuario las siguientes opciones:

- 1-Agregar país.
- 2-Actualizar país.
- 3-Buscar país.
- 4-Filtrar países.
- 5-Ordenar países.
- 6-Mostrar estadísticas.
- 7-Salir.

El menú permanece en ejecución mediante un ciclo `while` hasta que el usuario selecciona la opción de salida.

Además, se implementan validaciones para impedir el ingreso de opciones inválidas.

Registro de Países

La función `agregar_paises()` permite incorporar nuevos países al sistema.

El usuario debe indicar cuántos países desea registrar. Posteriormente los datos correspondientes a cada país son solicitados

Una vez ingresados los datos, se verifica que ninguno de los campos hayan sido dejados en blanco estos se guardan de forma permanente dentro del archivo `paises.csv`

```
-----  
Ingresar nombre del pais: China  
Ingresar superficie del pais: 150  
Ingresar poblacion del pais: 250  
Ingresar continente del pais: Asia  
-----  
  
Paises cargados correctamente!  
-----
```

Actualización de Información

La función `actualizar_paises()` permite modificar la información de un país previamente registrado.

El procedimiento consiste en:

1-Cargar los países desde el archivo.

2-Solicitar el nombre del país.

3-Buscar la coincidencia correspondiente.

4-Mostrar los datos actuales.

5-Permitir modificar:

Nombre.

Superficie.

Población.

Continente.

6-Reescribir el archivo con los datos actualizados.

Esta funcionalidad garantiza que la información almacenada permanezca actualizada.

```
-----  
Seleccione una opcion: 2  
  
Nombre del pais que desea actualizar: Brazil  
  
Pais encontrado: Brazil  
Superficie: 56465489  
Poblacion: 65166165  
Continente: America  
  
1. Nombre  
2. Superficie  
3. Poblacion  
4. Continente  
  
Que caracteristica desea actualizar?: 3  
Ingrese nuevo valor: 370  
  
Pais actualizado correctamente.  
-----
```

Búsqueda de Países

La función `buscar_pais()` permite localizar un país a partir de su nombre.

El programa recorre la lista de países cargada desde el archivo y compara el nombre ingresado por el usuario con los registros existentes.

Cuando encuentra una coincidencia, muestra toda la información asociada al país. Si dicha coincidencia no existe, se informa al usuario mediante el mensaje adecuado.

Si no existe una coincidencia exacta, el programa realiza una búsqueda parcial, mostrando todos los países cuyo nombre contiene el texto ingresado. Si tampoco se encuentra ninguna coincidencia, se informa al usuario mediante el mensaje "País no encontrado".

```
-----
Seleccione una opcion: 3

Ingresar nombre del pais (0 para volver al menu): Arge

No se encontro una coincidencia exacta. Resultados parciales para 'Arge':

Pais encontrado: Argentina
Superficie: 5165161
Poblacion: 56478946
Continente: America

Ingresar nombre del pais (0 para volver al menu):
```

Filtrado de Información

Filtrado por continente: Muestra todos los países pertenecientes al continente indicado.

Filtrado por rango de población: Muestra únicamente los países cuya población se encuentra dentro de un rango mínimo y máximo especificado por el usuario.

Filtrado por rango de superficie: Permite visualizar los países cuya superficie se encuentra comprendida dentro del rango solicitado.

```
-----
Seleccione una opcion: 4

1. Filtrar por Continente:
2. Filtrar por Rango de poblacion:
3. Filtrar por Rango de superficie:

Ingrese opcion para filtrar (0 para salir): 3
Ingrese rango minimo: 0
Ingrese rango maximo: 300
Países dentro del rango de superficie ingresado:

Corea del sur
Mongolia
China
-----
```


Ordenamiento de Datos

La función `ordenar_paises()` permite organizar los registros utilizando diferentes criterios.

Los ordenamientos disponibles son: por nombre, por población, por superficie.

Para el caso de la superficie se implementa además la posibilidad de ordenar en forma ascendente o descendente.

La operación se realiza mediante el método `sort()` y funciones lambda, lo que permite ordenar la lista de manera eficiente.

```

Ingrese opcion para ordenar (0 para salir): 1
Alemania - Superficie: 1654615 - Poblacion: 616516 - Continente: Europa
Argentina - Superficie: 5165161 - Poblacion: 56478946 - Continente: America
Brazil - Superficie: 56465489 - Poblacion: 370 - Continente: America
China - Superficie: 150 - Poblacion: 250 - Continente: Asia
Corea del sur - Superficie: 270 - Poblacion: 50 - Continente: Asia
Japon - Superficie: 131315 - Poblacion: 546132 - Continente: Asia
Mongolia - Superficie: 50 - Poblacion: 50 - Continente: Asia
-----

```

Estadísticas

La función `mostrar_estadisticas()` genera indicadores básicos a partir de los datos almacenados.

Entre las estadísticas implementadas se encuentran:

País con mayor y menor población: Se ordena la lista por población y se muestran los extremos del conjunto de datos.

Se utiliza un diccionario auxiliar para contabilizar cuántos países pertenecen a cada continente.

Estas estadísticas permiten obtener información resumida del dataset procesado.

```

Mostrar estadísticas:
1. País con mayor y menor población:
2. Promedio de población:
3. Cantidad de países por continente:
4. Promedio de superficie:

Ingrese una opción (0 para salir):1

El país con menor población es Corea del sur con 50 habitantes

El país con mayor población es Argentina con 56478946 habitantes

```

Promedio de población:

Se recorre la lista de países sumando la población de cada uno y luego se divide el total por la cantidad de países cargados, obteniendo así el promedio de habitantes del dataset.

```

Ingrese una opción (0 para salir):2

Promedio de habitantes: 8234616.29

Mostrar estadísticas:
1. País con mayor y menor población:
2. Promedio de población:
3. Cantidad de países por continente:
4. Promedio de superficie:

```

Promedio de superficie:

De manera similar, se suma la superficie de todos los países y se divide por la cantidad total de registros, obteniendo el promedio de superficie.

```

Mostrar estadísticas:
1. País con mayor y menor población:
2. Promedio de población:
3. Cantidad de países por continente:
4. Promedio de superficie:

Ingrese una opción (0 para salir):4

Promedio de superficie: 9059578.57 km²

```

Cantidad de países por continente: Se utiliza un diccionario auxiliar donde cada clave representa un continente y su valor la cantidad de países encontrados en él. Por cada país recorrido, se incrementa el contador correspondiente a su continente, mostrando finalmente el total por cada uno.

```
Mostrar estadísticas:  
1. País con mayor y menor población:  
2. Promedio de población:  
3. Cantidad de países por continente:  
4. Promedio de superficie:  
  
Ingrese una opción (0 para salir):4  
  
Promedio de superficie: 9059578.57 km²
```

Validaciones Implementadas

Con el objetivo de evitar errores durante la ejecución del programa se incorporaron diversas validaciones:

Control de opciones válidas dentro de los menús.

Verificación de campos vacíos.

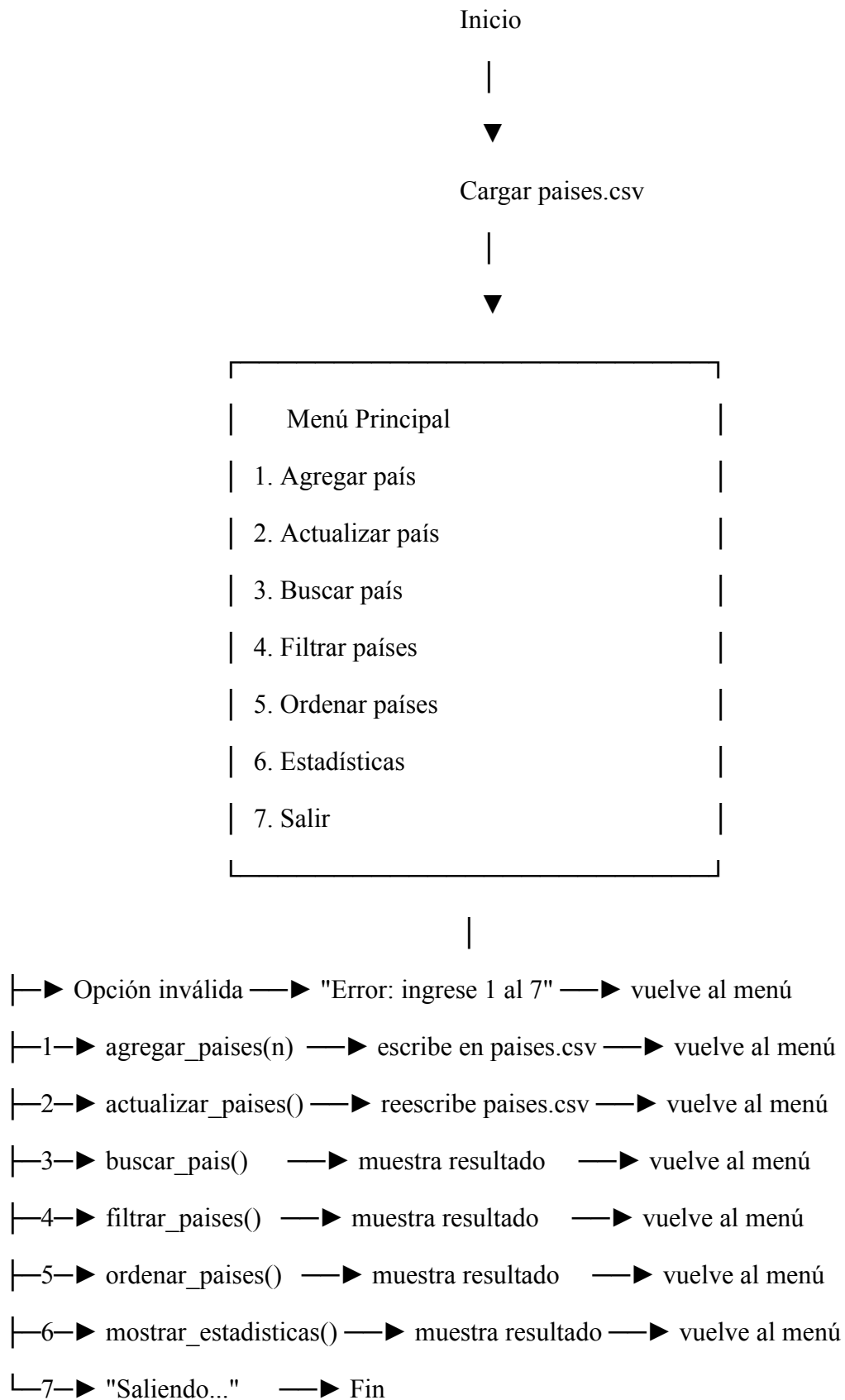
Manejo de excepciones mediante bloques try-except.

Control de conversiones numéricas.

Mensajes descriptivos para informar errores al usuario.

Estas validaciones mejoran la robustez y la experiencia de uso del sistema.

Diagrama de flujos principales



Dificultades y Conclusiones

Una de las principales dificultades que surgieron durante el desarrollo del proyecto fue la coordinación del trabajo en equipo dentro del repositorio de GitHub.

Para evitar conflictos en el código, se adoptó la práctica de trabajar en ramas individuales, donde cada integrante podría desarrollar su código de forma independiente sin interferir en el trabajo del compañero.

Una vez finalizada cada funcionalidad, se realizaba un Pull Request hacia la rama principal (main), permitiendo revisar los cambios antes de integrarlos.

Complementariamente, se intentó mantener una comunicación constante entre los integrantes para coordinar qué función cada uno trabajaría en cada momento, evitando así que ambos trabajaran los mismos archivos de forma simultánea.

Esta metodología de trabajo, si bien requirió una curva de aprendizaje inicial por tratarse de la primera experiencia utilizando Git de forma colaborativa, resultó sumamente valiosa ya que permitió mantener el código organizado y redujo significativamente la aparición de errores por sobreescritura.

Bibliografía / Webgrafía:

<https://docs.python.org/es/3/tutorial/datastructures.html>

<https://docs.python.org/es/3/tutorial/datastructures.html#dictionaries>

<https://docs.python.org/es/3/tutorial/controlflow.html#defining-functions>

<https://docs.python.org/es/3/tutorial/controlflow.html#if-statements>

<https://docs.python.org/es/3/howto/sorting.html>

<https://docs.python.org/es/3/library/statistics.html>

<https://docs.python.org/es/3/library/csv.html>

Links

Repositorio en Github:

<https://github.com/santiaguero91/UTN-TUPaD-Prog1-TPI>

Video explicativo de youtube:

<https://www.youtube.com/watch?v=vid01bQVI38>